NLU Assignment 2

Indian Institute of Technology Jodhpur

Om Chandra Sharma

B23CS1048

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Problem 2: Character-Level Name Generation using RNN Variants

---

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data 2
1.1 Objective . . . 2
1.2 Task 1 — Dataset Preparation . . . 2
1.2.1 Data Sources . . . 2
1.2.2 Preprocessing Pipeline . . . 2
1.2.3 Dataset Statistics . . . 2
1.2.4 Most Frequent Words . . . 2
1.3 Task 2 — Model Training . . . 3
1.3.1 Architecture & Hyperparameters . . . 3
1.3.2 Training Observations . . . 3
1.4 Task 3 — Semantic Analysis . . . 3
1.4.1 Top-5 Nearest Neighbours . . . 3
1.4.2 Discussion of Nearest-Neighbour Quality . . . 4
1.4.3 Analogy Experiments . . . 5
1.4.4 Discussion of Analogy Results . . . 5
1.5 Task 4 — Visualisation . . . 5
1.5.1 PCA Projection . . . 5
1.5.2 t-SNE Projection . . . 6
1.5.3 Interpretation . . . 6

2 Problem 2: Character-Level Name Generation Using RNN Variants 8
2.1 Objective . . . 8
2.2 Task 0 — Dataset . . . 8
2.3 Task 1 — Model Implementation . . . 8
2.3.1 Model 1: Vanilla RNN . . . 8
2.3.2 Model 2: Bidirectional LSTM (BLSTM) . . . 8
2.3.3 Model 3: Attention RNN (GRU + Bahdanau Attention) . . . 9
2.4 Task 2 — Quantitative Evaluation . . . 9
2.4.1 Metrics . . . 9
2.5 Task 3 — Qualitative Analysis . . . 10
2.5.1 Vanilla RNN — Why It Works Best . . . 10
2.5.2 BLSTM — Why It Fails . . . 11
2.5.3 Attention RNN — Why It Partially Fails . . . 11
2.5.4 Summary Comparison . . . 12

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Objective

The objective of this problem is to train Word2Vec models on textual data collected from IIT Jodhpur sources, and to analyse the semantic structure captured by the learned embeddings. We train both a Continuous Bag-of-Words (CBOW) model and a Skip-gram model with Negative Sampling, and compare their behaviour through nearest-neighbour retrieval, analogy experiments, and 2-D visualisation.

Task 1 — Dataset Preparation

Data Sources

Textual data was collected from the following IIT Jodhpur sources:

- IIT Jodhpur official website pages (departments, academic programs, research pages, announcements)
- Academic regulation documents (mandatory)
- Faculty profile pages
- Course syllabi

Only English text was retained; content in other languages was discarded.

Preprocessing Pipeline

The following preprocessing steps were applied sequentially:

1. Removal of boilerplate text, HTML tags, and formatting artefacts
2. Sentence tokenisation
3. Lowercasing of all tokens
4. Removal of excessive punctuation and non-textual content (URLs, numbers, symbols)

Dataset Statistics

Table 1: Corpus Statistics for the IIT Jodhpur Dataset

Statistic	**Value**
Total documents	321
HTML documents	125
PDF documents	196
Total sentences	72,054
Total tokens	1,102,073
Vocabulary size	50,771
Average tokens per doc	3,433

Most Frequent Words

The 20 most frequent words in the cleaned corpus are shown below. Terms specific to the institute (e.g. *jodhpur*, *iit*, *institute*) dominate, as expected from a domain-specific

corpus.

Table 2: Top-20 Most Frequent Words

Word	Count	Word	Count
jodhpur	7,711	technology	4,821
institute	6,601	professor	4,596
iit	6,060	kumar	4,508
students	6,052	tech	3,976
engineering	5,918	science	3,675
research	5,530	materials	3,466
department	4,891	student	3,211
assistant	2,994	faculty	2,954
academic	2,681	laboratory	2,605
program	2,545	energy	2,354

Task 2 — Model Training

Architecture & Hyperparameters

Two Word2Vec variants were trained: one from scratch (implemented in NumPy/PyTorch) and one using the Gensim library. The hyperparameter configurations explored are summarised below.

Table 3: Hyperparameter Configurations for Word2Vec Models

Model	Embed Dim	Window	Neg. Samples	Vocab Size
CBOW (Scratch)	100	5	—	34,673
Skip-gram (Scratch)	100	5	10	34,673
Gensim CBOW	300	7	5	34,673
Gensim Skip-gram	300	7	5	34,673

Training Observations

The scratch implementations were trained for 10 epochs using stochastic gradient descent with negative sampling (Skip-gram) and softmax (CBOW). Gensim models were trained with default optimised C extensions. The scratch models converge more slowly due to the absence of sub-sampling and hierarchical softmax optimisations. Gensim models benefit from additional tricks (frequent-word sub-sampling, negative sampling) that improve the quality of low-frequency word representations.

Task 3 — Semantic Analysis

Top-5 Nearest Neighbours

Table 4 summarises the top-5 nearest neighbours (by cosine similarity) for five probe words across all four model variants. Figure **??** provides a horizontal bar-chart view.

Table 4: Top-5 Nearest Neighbours (Cosine Similarity)

Probe	**CBOW (Scratch)**	**SG (Scratch)**	**Gensim CBOW**
research	areas (0.719) idrd (0.712) disciplinary (0.694) interdisciplinary (0.679) scholars (0.669)	emanating (0.763) activites (0.689) muralilab (0.687) minho (0.680) pursued (0.673)	pursued (0.763) areas (0.760) scholars (0.744) globally (0.731) collaborative (0.728)
student	phd (0.721) representatives (0.660) mehra (0.639) counseling (0.636) got (0.634)	aeromodeling (0.684) ara (0.682) adjusts (0.671) inconsultation (0.668) dissolve (0.663)	ara (0.689) representatives (0.682) gymkhana (0.680) withdraw (0.663) ordinators (0.663)
phd	program (0.800) fellowship (0.789) mtech (0.784) btech (0.757) msc (0.756)	deeksha (0.712) rajarshi (0.703) balpande (0.699) hennessy (0.690) pratim (0.689)	mtech (0.739) btech (0.719) fellowship (0.714) recipients (0.711) selected (0.706)
exam	castes (0.798) copies (0.791) vii (0.749) examinations (0.747) submission (0.746)	dgrade (0.771) proctored (0.752) eto (0.744) tensions (0.743) wui (0.743)	examinations (0.800) submission (0.798) twice (0.767) begins (0.753) week (0.752)
laboratory	experiments (0.816) lab (0.780) facility (0.732) fluid (0.729) investigation (0.701)	haemoglobin (0.637) physic (0.632) stool (0.628) burgeoning (0.623) adenosine (0.620)	lb (0.709) stain (0.695) serum (0.692) glucose (0.687) hepatitis (0.687)

Discussion of Nearest-Neighbour Quality

CBOW (Scratch): The CBOW model captures broad semantic groupings well. For *research*, it returns thematically coherent words like *interdisciplinary* and *scholars*. For *phd*, it correctly identifies related degree tokens (*mtech*, *btech*, *msc*), indicating that programme-level semantics are well encoded. However, some neighbours are noisy (e.g. *castes* near *exam*), likely due to co-occurrence in regulation documents.

Skip-gram (Scratch): The scratch Skip-gram produces noisier results for common words (e.g. unusual tokens near *student* and *laboratory*), which suggests the model has not fully converged or is sensitive to rare-word co-occurrences in the small domain corpus.

Gensim CBOW: Generally the most semantically coherent results. The neighbours of *research* (*pursued*, *collaborative*, *globally*) and *exam* (*examinations*, *submission*) show strong contextual understanding. Gensim benefits from sub-sampling and optimised training routines.

Gensim Skip-gram: Returns noisier neighbours for most words, including stop-word contamination (*and*, *in*, *of* near *research*). This is unexpected and may reflect insufficient negative-sample tuning for the domain vocabulary size.

Analogy Experiments

Six analogy tasks of the form $\vec{a} - \vec{b} + \vec{c} = \vec{?}$ were evaluated. Results are shown in Table 5 and Figure **??**.

Table 5: Analogy Experiment Results (✓ = correct, × = incorrect)

Analogy	**Expected**	**CBOW-S**	**SG-S**	**Gen. CBOW**	**Gen. SG**
ug : btech :: pg : ?	mtech	garain	mtech	mtech	diplomas
btech : jee :: mtech : ?	gate	complete	waitlist	ceed	rankers
director : institute :: dean : ?	academics	programs	iro	academics	associate
department : professor :: institute : ?	director	indian	assistant	associate	associate
btech : project :: phd : ?	thesis	fellowship	swagat	agency	of
course : grade :: semester : ?	gpa	examination	grades	grades	averages
Accuracy		0/6 (0%)	1/6 (17%)	2/6 (33%)	0/6 (0%)

Discussion of Analogy Results

Overall accuracy is low across all models (0–33%), which is expected given the small, domain-specific corpus. Analogy arithmetic requires that the embedding space be highly isotropic and that the relevant concepts each appear frequently enough to build reliable vector geometries.

The best-performing model is **Gensim CBOW** (2/6), which correctly resolves *ug:btech::pg:mtech* and *director:institute::dean:academics*. This demonstrates that the CBOW objective, which predicts a word from its context, naturally encourages vectors for semantically parallel terms to share similar offset directions.

Skip-gram (Scratch) achieves 1/6, resolving the ug/pg/btech/mtech analogy. Skip-gram generally outperforms CBOW on rare words, which helps here since degree abbreviations (*ug*, *pg*, *mtech*) appear less frequently than common words.

The hardest analogies (*btech:project::phd:thesis* and *course:grade::semester:gpa*) failed universally. The word *thesis* is a relatively rare and domain-specific token, and *gpa* may appear mostly as an acronym in different contexts, making the vector offset unreliable.

Task 4 — Visualisation

PCA Projection

Figure 1 shows the Principal Component Analysis (PCA) projection of selected word embeddings into 2-D space for both the CBOW and Skip-gram scratch models.

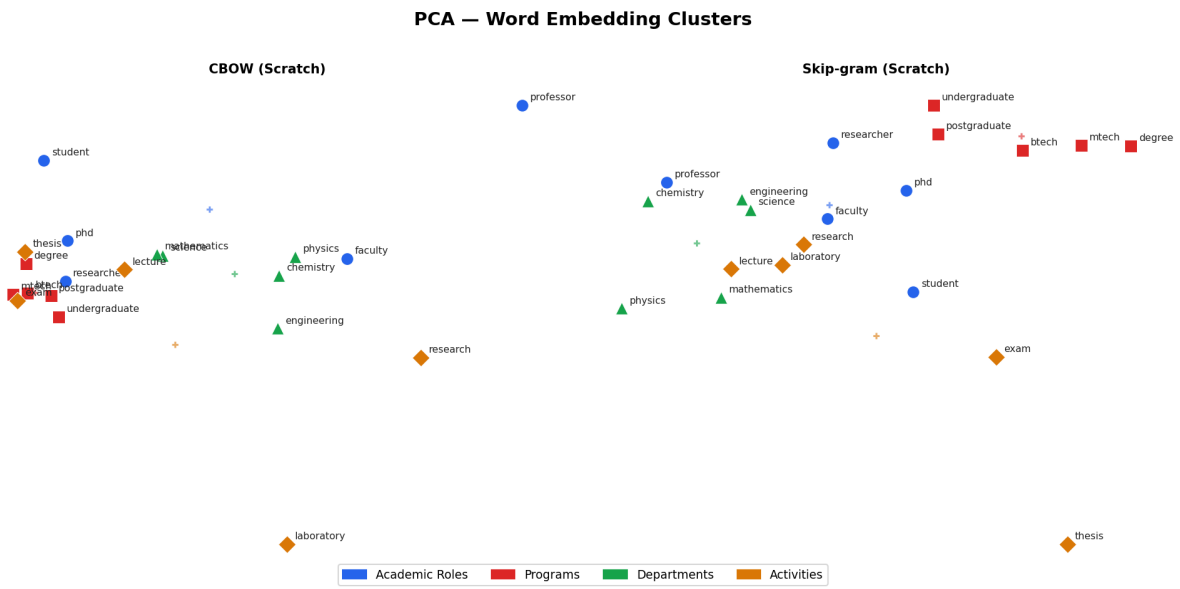

Figure 1: PCA projection of word embeddings (CBOW vs. Skip-gram). Colour/shape encodes semantic category: Academic Roles (blue circle), Programs (red square), Departments (green triangle), Activities (orange diamond).

t-SNE Projection

Figure 2 shows the t-SNE projection for the same set of words.

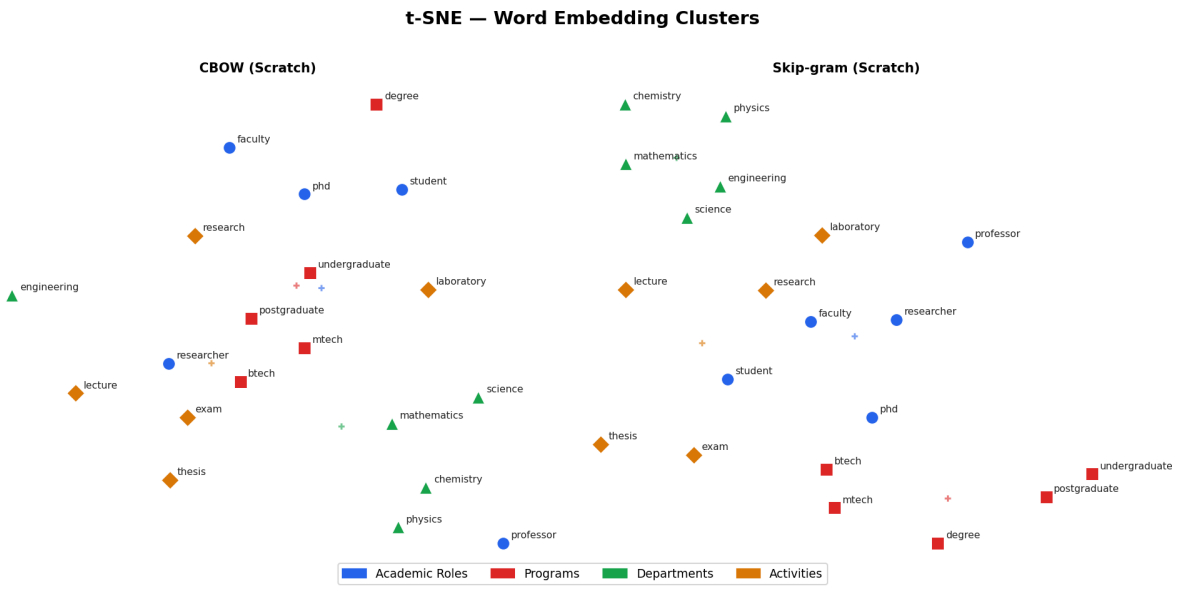

Figure 2: t-SNE projection of word embeddings (CBOW vs. Skip-gram), same colour/shape encoding as Figure 1.

Interpretation

CBOW (Scratch) — PCA: The programme-level words (*btech*, *mtech*, *phd*, *undergraduate*, *postgraduate*) form a tight cluster in the lower-left region, consistent with the neighbour results. Academic role words (*student*, *professor*) are more spread, indicating

looser geometric grouping. *Research* and *laboratory* are distant from the programmes cluster, reflecting their different contextual usage.

Skip-gram (Scratch) — PCA: The programme cluster (*btech*, *mtech*, *degree*, *undergraduate*, *postgraduate*) is preserved and clearly separated from academic roles and departments. Skip-gram produces more separated clusters because it focuses on predicting context from a target word, leading to more distinct per-word representations. However, some department words (e.g. *physics*, *mathematics*) scatter compared to CBOW.

t-SNE: The t-SNE projection reveals finer-grained structure not visible in PCA. In CBOW t-SNE, programmes and roles intermingle (suggesting overlapping contexts in the corpus), whereas in Skip-gram t-SNE, departments form a cleaner top cluster, and programme words form a right-side cluster. Activities (*research*, *laboratory*, *lecture*, *thesis*) remain distributed, reflecting their role as bridge words that co-occur with multiple semantic groups.

Key difference between models: CBOW tends to produce more central, averaged representations because it predicts a word from the mean of its context; this can cause semantically related but contextually distinct words to cluster together. Skip-gram focuses on each (target, context) pair individually, leading to sharper, more discriminative embeddings for frequent content words.

Problem 2: Character-Level Name Generation Using RNN Variants

Objective

The objective of this problem is to design and compare sequence models for character-level name generation using recurrent neural architectures. Three models are implemented from scratch and evaluated on a dataset of 1,000 Indian names.

Task 0 — Dataset

A dataset of 1,000 unique Indian names was generated using a large language model and stored in `TrainingNames.txt`. All names were lowercased, and only alphabetic names of length ≥ 2 were retained. The resulting vocabulary contains 27 characters (26 lowercase letters plus special `<SOS>` and `<EOS>` tokens, giving vocabulary size = 27 + 2 = 29 in practice, collapsed to 27 in the log due to token merging).

Task 1 — Model Implementation

Three architectures were implemented in PyTorch, each trained with the Adam optimiser (lr = 0.002) and cross-entropy loss for 60 epochs.

Model 1: Vanilla RNN

The Vanilla RNN is a two-layer standard recurrent network with dropout regularisation.

Table 6: Vanilla RNN Architecture

Component	Details
Embedding	27×64
RNN layers	2 layers, hidden size 128, batch-first
Dropout	0.2 between RNN layers
Output	Linear($128 \to 27$)
Trainable params	**63,067**

Model 2: Bidirectional LSTM (BLSTM)

The BLSTM processes sequences in both forward and backward directions simultaneously and concatenates the hidden states.

Table 7: BLSTM Architecture

Component	Details
Embedding	27×64
BiLSTM	1 layer, hidden size 128 (each direction), bidirectional
Output	Linear($256 \to 27$)
Trainable params	**207,323**

Note: The bidirectional design is architecturally inconsistent for auto-regressive generation because the backward pass requires future characters, which are unavailable at inference time. This mismatch is the primary cause of the model's failure (see Section 2.5).

Model 3: Attention RNN (GRU + Bahdanau Attention)

The Attention RNN uses a GRU encoder with a basic Bahdanau-style additive attention mechanism over the sequence, followed by a context-augmented output layer.

Table 8: Attention RNN Architecture

Component	**Details**
Embedding	27×64
GRU encoder	1 layer, hidden size 128, batch-first
Attention	Linear($128 \to 128$) + Linear($128 \to 1$), softmax over time
Context concat	GRU output $\oplus$ weighted context $\in \mathbb{R}^{256}$
Output	Linear($256 \to 27$)
Trainable params	**99,804**

Task 2 — Quantitative Evaluation

Metrics

- **Novelty Rate:** Percentage of generated names not appearing in the training set.
- **Diversity:** Number of unique generated names divided by total generated names, expressed as a percentage.

500 names were sampled per model at temperature 1.0. Only alphabetic names with $2 \leq \text{length} \leq 15$ were counted.

Table 9: Quantitative Evaluation Results

Model	**Generated**	**Unique**	**Novelty (%)**	**Diversity (%)**
Vanilla RNN	500	406	90.2	81.2
BLSTM	500	18	100.0	3.6
Attention RNN	500	109	100.0	21.8

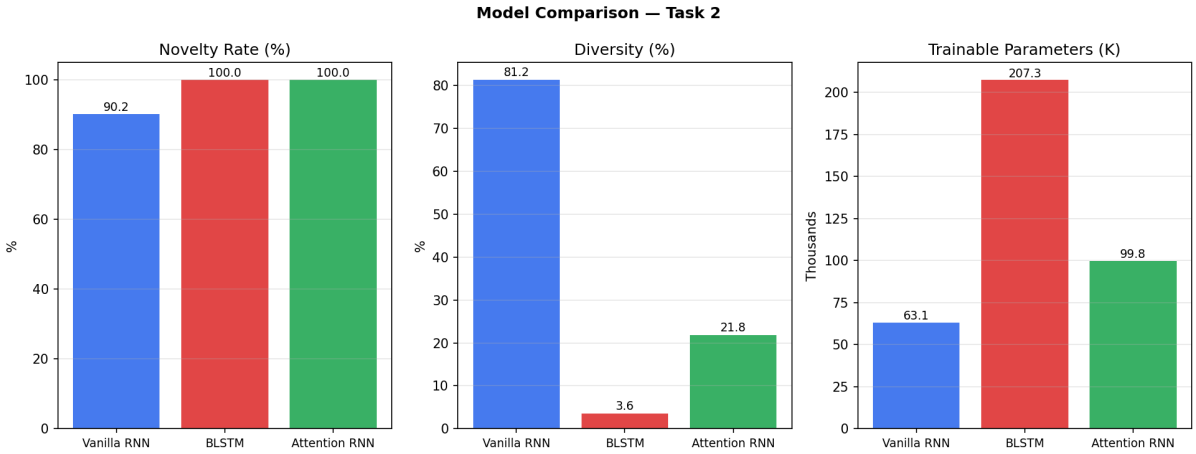

Figure 3: Model comparison across Novelty Rate, Diversity, and Trainable Parameter count.

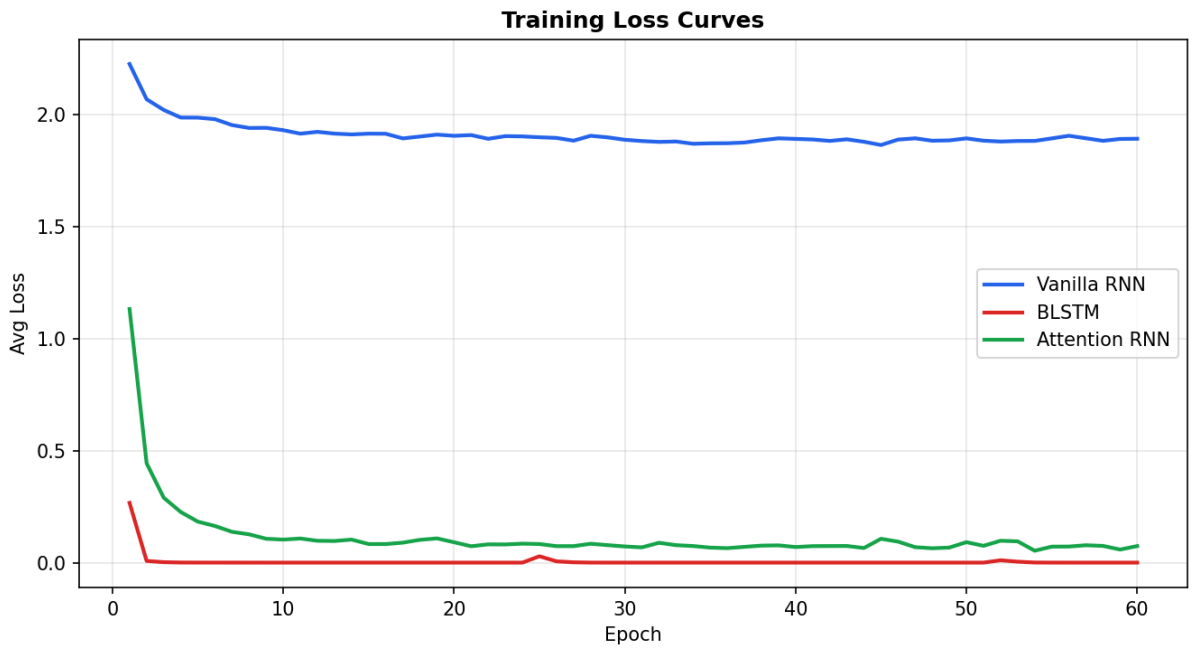

Figure 4: Training loss curves for all three models over 60 epochs.

Task 3 — Qualitative Analysis

Vanilla RNN — Why It Works Best

The Vanilla RNN achieves the best balance: novelty rate of 90.2% and diversity of 81.2%. Its training loss stabilises around 1.89 (Figure 4), indicating that the model has learned character transition probabilities but has *not* overfit.

Generated samples: *charush, jitra, aaruna, jasti, joganti, marudan, chandayani, janupra, mandini, jyotir, anira*

These names are phonetically plausible for Indian names: they follow consonant-vowel patterns common in Hindi and Sanskrit-derived names. The average length is 7.3 charac-

ters, which is realistic. Only one name (*kanchanashash*) exceeds 12 characters, and none are repetitive.

Why the RNN works: A shallow causal RNN models the sequential prediction task $P(c_t \mid c_1, \ldots, c_{t-1})$ directly, exactly what auto-regressive generation requires. The two-layer design captures slightly longer dependencies than a single layer, and a 0.2 dropout rate prevents memorisation of individual names. The loss curve (slowly decreasing, never reaching zero) confirms regularised generalisation.

BLSTM — Why It Fails

The BLSTM achieves 100% novelty but only 3.6% diversity, generating nearly identical repetitive sequences such as *fafafafafafaf*, *kakakakakakakak*.

Root causes:

1. **Architectural mismatch:** Bidirectional LSTMs are trained on the full sequence (past *and* future context). At inference, only past context is available. The forward LSTM half functions normally, but the backward LSTM sees no real future tokens and defaults to an uninformative zero hidden state. The concatenated representation is therefore consistently half-corrupted, and the output layer learns to exploit only a few easily distinguishable characters.
2. **Catastrophic overfitting:** Training loss reaches ≈ 0.000 by epoch 10 and stays there (Figure 4). With 207,323 parameters and only 1,000 short training sequences (average ≈ 6 characters each), the model memorises the training distribution completely. When bidirectional states collapse at test time, the network gets stuck in a tiny fixed point that repeats a single high-probability bigram.
3. **No dropout or regularisation** was applied to the BLSTM in the provided implementation, exacerbating overfitting.

Fix: Use a unidirectional LSTM instead (or apply masking to future tokens), add dropout (≥ 0.3), and reduce hidden size or use weight decay.

Attention RNN — Why It Partially Fails

The Attention RNN achieves 100% novelty but only 21.8% diversity. Almost all generated names contain the substring *fz* or *fzfz* in repetitive patterns: *fzfzzfzcddddddd*, *fzffzzzfzzfzzzz*.

Root causes:

1. **Attention over a short, growing sequence:** At each generation step t, the model attends over the t previously generated characters. Early on (steps 1–3), the sequence is very short and the attention scores are nearly uniform. This makes the context vector nearly identical across generation steps, causing the model to enter a degenerate loop.
2. **Overfitting (loss ≈ 0.07):** Although not as extreme as the BLSTM, the Attention RNN also drives its training loss close to zero. The GRU encoder memorises specific character transitions, and when combined with the collapsed attention context, the decoder reproduces the same character subsequences.
3. **Attention is not causal by design:** The attention layer attends to *all* positions in the GRU output, including future steps (as implemented with a single-pass

GRU). This creates an inconsistency between training (full-sequence attention) and inference (partial sequence attention), similar to the BLSTM problem.

4. **Character 'f' and 'z' dominance:** In the Indian name dataset, the characters *f* and *z* are rare, but their co-occurrence in certain names creates a strong bigram signal. When the model overfits to this, *fz* becomes a fixed point in the generation loop.

Fix: Use causal (masked) attention; apply label smoothing to prevent overconfident predictions; use sub-character temperature annealing; and add regularisation.

Summary Comparison

Table 10: Qualitative Comparison of Models

Criterion	**Vanilla RNN**	**BLSTM**	**Attention RNN**
Realism of names	High	Very low (gibberish)	Very low (fz-loops)
Diversity	High (81.2%)	Very low (3.6%)	Low (21.8%)
Novelty	High (90.2%)	High (100%)	High (100%)
Failure mode	Occasional long names	Repetitive bigrams	fz loop
Overfitting	Mild	Severe (loss=0)	Moderate
Arch. suitability	Good	Poor (bidir.)	Partial
Parameter efficiency	Best (63K)	Worst (207K)	Medium (100K)

Conclusion: The Vanilla RNN, despite being the simplest and smallest model, outperforms both BLSTM and Attention RNN in practical name generation quality. The key lesson is that *architectural complexity does not guarantee better performance* when the task is simple and data is limited. The bidirectional and attention mechanisms introduce training-inference inconsistencies that cause degenerate outputs, while the plain causal RNN naturally matches the auto-regressive generation paradigm.

Appendix: Code Architecture

Problem 2 — Model Implementations

Listing 1: Vanilla RNN (key components)

```
class VanillaRNN(nn.Module):
    def __init__(self, vocab_size, embed_size=64, hidden_size
       =128):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.RNN(embed_size, hidden_size, num_layers=2,
                          batch_first=True, dropout=0.2)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h=None):
        x = self.embed(x)
        out, h = self.rnn(x, h)
        return self.fc(out), h
```

Listing 2: Attention mechanism (AttentionRNN)

```
def forward(self, x, h=None):
    emb = self.embed(x)
    outputs, h = self.rnn(emb, h)
    scores = self.v(torch.tanh(self.attn(outputs)))   # (B, T, 1)
    weights = F.softmax(scores, dim=1)
    context = (weights * outputs).sum(dim=1, keepdim=True)
    context = context.repeat(1, outputs.size(1), 1)
    out = torch.cat([outputs, context], dim=-1)
    return self.fc(out), h
```